Hidden Device Heterogeneity in Constrained ML Deployment: How Dynamic Quantization Creates Stochastic Feasibility Boundaries

Christian Lysenstøen
UC Berkeley
University of Inland Norway
christian@lysenstoen.net

***Abstract*—Deploying machine learning models under latency constraints requires evaluating candidate configurations against strict feasibility thresholds. We present an empirical study revealing that deployment search spaces contain a hidden device heterogeneity: PyTorch's dynamic INT8 quantization silently moves inference from GPU to CPU, fundamentally altering latency distribution characteristics. Through 2,150 controlled measurement trials across 25 configurations on an NVIDIA RTX 5080, we show that GPU-resident configurations (FP32, FP16) exhibit near-deterministic feasibility—14 configurations tested with essentially zero feasibility flips. In stark contrast, CPU-resident configurations (INT8 dynamic) show feasibility flip rates up to 39%, with MobileNetV2 at batch size 3 producing a mean p95 latency of 19.70 ± 3.46 ms against a 20 ms threshold—a standard deviation of 17.6% of the mean. This stochasticity is not explained by any single observable system metric (CPU frequency, CPU utilization, GPU temperature), but arises from irreducible OS scheduling jitter affecting CPU-bound inference. The effect is architecture-dependent: MobileNetV2's depthwise separable convolutions under dynamic quantization create many small, memory-bandwidth-bound CPU kernels that accumulate scheduling jitter, while ResNet-18's larger fused kernels remain stable. Cross-GPU evaluation confirms that the entire feasibility landscape shifts across hardware ($5.8\times$ latency ratio for identical configurations). We show that fixed safety margins—the obvious practitioner response—fail because boundary stochasticity is heterogeneous across execution devices: margins wide enough for CPU-resident volatile configurations over-constrain deterministically stable GPU-resident ones. Our findings imply that deployment optimizers should be device-aware, distinguishing GPU-resident from CPU-resident configurations in both evaluation protocol and feasibility assessment.**

***Index Terms*—ML deployment, quantization, constrained optimization, CPU inference, latency variability, feasibility, benchmarking**

I. Introduction

Deploying machine learning models to production requires solving a constrained optimization problem: find a deployment configuration—model architecture, inference backend, quantization scheme, and batch size—that maximizes task accuracy while satisfying hardware constraints on latency and memory [1].

A fundamental, implicit assumption in this process is that all configurations in the search space execute on the same device. When a practitioner specifies "run inference on this GPU," they expect every configuration—regardless of quantization mode—to execute on that GPU. Deployment optimizers like Optuna [2] and TBA [1] treat the search space as homogeneous: every configuration is evaluated once, on the same device, with the same measurement protocol.

We show this assumption is violated in practice.

PyTorch's `quantize_dynamic` API—the standard path for INT8 deployment—silently moves inference from GPU to CPU [5]. The model is transferred to CPU memory, quantized operators execute on the CPU, and timing characteristics change fundamentally. This creates a *hidden device heterogeneity* within the deployment search space: some configurations run on GPU with CUDA-event-level timing precision and near-deterministic latency, while others run on CPU with wall-clock timing and high scheduling jitter. No deployment optimizer we are aware of accounts for this distinction.

The practical consequence is severe for configurations near a constraint boundary. We demonstrate that:

1) **GPU-resident inference is deterministic.** Across 14 GPU-resident configurations evaluated with 50+ repeats each, GPU-resident (FP32/FP16) configurations show essentially zero feasibility flips at the 20 ms latency threshold.
2) **CPU-resident inference is stochastic.** MobileNetV2 under INT8 dynamic quantization exhibits a 39% flip rate—the same configuration, same hardware, same inputs, but different feasibility outcomes across runs due to OS scheduling noise.
3) **The stochasticity is architecture-dependent.** Depthwise separable convolutions (MobileNetV2) produce high variance under CPU-bound INT8 inference, while standard convolutions (ResNet-18) remain stable. This heterogeneity means no single safety margin is appropriate.
4) **Feasibility does not transfer across GPUs.** The same configuration runs at 19 ms on an RTX 5080 and 110 ms

on a Tesla T4—a $5.8\times$ shift that completely relocates the feasibility boundary.

II. Background and Problem

A. Constrained Deployment Optimization

Let $\mathcal{X}$ denote the space of deployment configurations $\mathbf{x} = (\text{model}, \text{backend}, \text{quantization}, \text{batch_size})$. The deployment optimization problem is:

$$\max_{\mathbf{x}\in\mathcal{X}} f(\mathbf{x}) \quad \text{s.t.} \quad g_i(\mathbf{x}) \le \tau_i, \quad i = 1, \ldots, m \tag{1}$$

where $f(\mathbf{x})$ is task accuracy, $g_i(\mathbf{x})$ are constraint functions (p95 latency, peak memory), and τ_i are thresholds. Existing frameworks assume $g_i(\mathbf{x})$ returns a fixed value for any $\mathbf{x}$, making feasibility deterministic:

$$\text{feasible}(\mathbf{x}) = \prod_{i=1}^{m} \mathbb{1}[g_i(\mathbf{x}) \le \tau_i] \in \{0, 1\} \tag{2}$$

B. The Hidden Device Switch

PyTorch's dynamic INT8 quantization operates exclusively on CPU [5]. Internally, `quantize_dynamic` dispatches to the FBGEMM backend, which provides optimized CPU-only INT8 operators for `nn.Linear` and `nn.Conv2d` modules. The model is moved to CPU memory and no GPU kernel is launched for these configurations. The calling code receives no warning that the execution device has changed.

In a deployment search space containing FP32, FP16, and INT8 configurations, this means the search space is *mixed-device*:

$$\text{device}(\mathbf{x}) = \begin{cases} \text{GPU} & \text{if } \mathbf{x}.\text{quant} \in \{\text{fp32}, \text{fp16}\} \\ \text{CPU} & \text{if } \mathbf{x}.\text{quant} = \text{int8_dynamic} \end{cases} \tag{3}$$

This device assignment is invisible to the optimizer, which treats all configurations identically. Practitioners can detect this by inspecting `next(model.parameters()).device` after quantization—if it returns `cpu` when GPU inference was intended, the silent device switch has occurred.

C. Why Device Matters for Feasibility

GPU inference benefits from deterministic kernel execution: CUDA kernels are scheduled by the GPU's own hardware scheduler, largely isolated from OS-level interference. Latency measurements via `torch.cuda.Event` capture precise GPU execution time.

CPU inference is subject to OS process scheduling, interrupt handling, context switching, power state transitions, and memory bus contention. Measurements via `time.perf_counter()` capture wall-clock time including all system interference. For workloads with many small kernels (as in depthwise separable convolutions), these effects accumulate.

We define the *feasibility probability* of a configuration as:

$$\phi(\mathbf{x}) \triangleq P[g(\mathbf{x}) \le \tau] \tag{4}$$

where the probability is over the distribution of system states during evaluation.

Definition 1 (Flip Rate): The flip rate of a configuration is $\rho(\mathbf{x}) = \min(\phi(\mathbf{x}), 1 - \phi(\mathbf{x}))$. A flip rate of 0 indicates deterministic feasibility; 0.5 indicates maximum stochasticity.

Our central empirical question is: does the execution device (GPU vs. CPU) predict $\rho(\mathbf{x})$?

III. Experimental Setup

A. DeployBench Configuration Space

We use DeployBench [1], which evaluates inference configurations on real hardware: 5 models (ResNet-18, ResNet-50, MobileNetV2, EfficientNet-B0, ViT-Tiny), 3 backends (PyTorch Eager, `torch.compile`, ONNX Runtime), 3 quantization modes (FP32, FP16, INT8 Dynamic), and batch sizes 1–32. The constraint scenario `edge_tight` specifies $\tau_{\text{latency}} = 20\,\text{ms}$ (p95) and $\tau_{\text{memory}} = 512\,\text{MB}$. The dataset is ImageNette [7] (10-class ImageNet subset, 500 validation images).

B. Hardware

Primary: NVIDIA RTX 5080 (laptop), Windows, Python 3.14, PyTorch 2.x. Secondary: Tesla T4 (Google Colab), Linux.

C. Measurement Protocol

Each evaluation consists of 20 warmup passes (discarded) followed by 200 timed inference passes. We record the full latency vector and compute $g(\mathbf{x}) = t_{95}$. Timing is device-appropriate: GPU-resident configurations use `torch.cuda.Event` with CUDA synchronization; CPU-resident configurations use `time.perf_counter()`. At each repeat, we record GPU state via `nvidia-smi` and, for CPU-resident configurations, CPU frequency and utilization via `psutil`. Each configuration receives 50–250 repeats, alternating cold and warm conditions with interleaving to prevent temporal confounding.

D. Configuration Selection

We selected 25 configurations near the 20 ms boundary from prior runs. Of these, 5 crashed consistently, leaving 20 with valid data: 14 GPU-resident and 6 CPU-resident. See Section A for full accounting.

IV. Device-Dependent Feasibility Stochasticity

A. GPU-Resident Configurations Are Deterministic

TABLE I: Feasibility Stability by Execution Device

Device	Configs	$\rho > 0$	$\rho > 10\%$	Max ρ
GPU (FP32/FP16)	14	1	0	2%
CPU (INT8 dyn.)	6	3	2	**39%**

The single GPU flipper (ViT-Tiny/FP16/bs23 at $\rho = 2\%$) represents 1 disagreeing repeat out of 50—likely measurement noise. GPU temperature correlates with GPU-resident latency at $r = 0.677$ (pooled), but this is practically irrelevant because

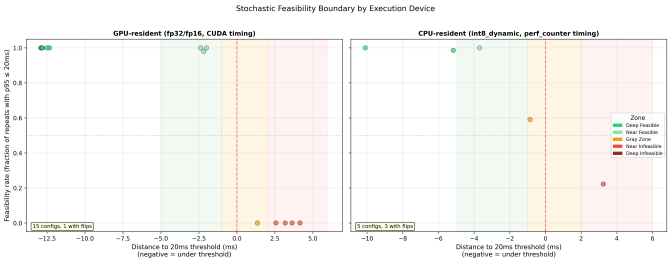

Fig. 1: Feasibility probability vs. distance to threshold, split by execution device. **Left**: GPU-resident configs show a sharp step function. **Right**: CPU-resident configs show a gradual transition with intermediate feasibility rates.

no GPU configuration's variance is large enough to cross the threshold.

B. CPU-Resident Configurations Are Stochastic

TABLE II: CPU-Resident Configuration Flip Rates (INT8 Dynamic)

Model	bs	$\bar{g} \pm \sigma$ (ms)	ϕ	ρ
MobileNetV2	3	19.70 ± 3.46	0.61	**39%**
MobileNetV2	4	22.79 ± 3.88	0.28	**28%**
EfficientNet-B0	1	$\sim$19.2 $\pm$ 1.1	0.99	1.3%
ResNet-18	1	$<$20 $\pm$ $\sim$0.5	1.00	0.0%

MobileNetV2/INT8 Dynamic at batch size 3 has $\sigma = 3.46$ ms (**17.6% of mean**). With mean p95 of 19.70 ms against a 20 ms threshold, OS scheduling noise pushes latency across the boundary on any given evaluation.

C. No Single Metric Explains CPU Stochasticity

CPU frequency reports a constant 2700 MHz (Windows base clock); CPU utilization shows $r = -0.09$ (bs3), $r = +0.02$ (bs4); GPU temperature shows $r \approx -0.02$ for CPU configs. The stochasticity arises from irreducible OS-level scheduling noise—context switches, interrupt handling, memory bus contention—that collectively produce high variance for CPU-bound workloads with many small operations. Windows does not expose dynamic CPU frequency or die temperature via standard APIs; Linux instrumentation may enable finer attribution in future work.

V. Architecture-Dependent Stochasticity

The flip rate varies across architectures under identical INT8 dynamic quantization: MobileNetV2 at 39%, EfficientNet-B0 at 1.3%, ResNet-18 at 0.0%.

A. Depthwise Separable Convolutions Amplify Jitter

MobileNetV2 [6] uses depthwise separable convolutions, creating many small, memory-bandwidth-bound CPU operations under INT8 quantization. With N sequential small kernels each experiencing scheduling jitter ϵ_j with variance σ_ϵ^2:

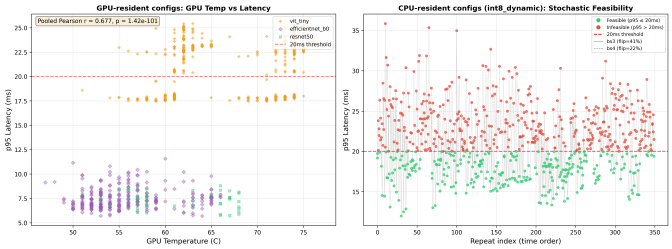

Fig. 2: **Left**: GPU-resident configs show thermal correlation ($r = 0.677$) but nothing crosses the threshold. **Right**: CPU-resident flippers show latency bouncing across the 20 ms line over successive repeats. Green = feasible, red = infeasible.

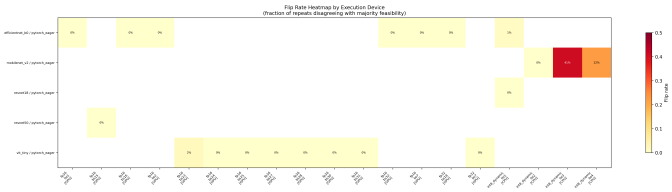

Fig. 3: Flip rate heatmap annotated with [GPU] or [CPU]. High flip rates are exclusively in CPU-resident INT8 dynamic configurations.

$$\mathrm{Var}[g_{\text{total}}] \approx N \cdot \sigma_\epsilon^2 + \mathrm{Var}[g_{\text{compute}}] \tag{5}$$

ResNet-18 fuses into fewer, larger kernels where jitter is a negligible fraction of execution time.

B. Heterogeneity Implications

The gray zone width depends on the architecture $\times$ quantization $\times$ device interaction: MobileNetV2/INT8 $\sim$4 ms, EfficientNet-B0/INT8 $<$1 ms, ResNet-18/INT8 $\approx$0 ms. This heterogeneity is why fixed margins fail (Section VII).

VI. Cross-GPU Analysis

TABLE III: Cross-GPU: MobileNetV2 / INT8 Dynamic

Config	GPU	p95 (ms)	ϕ	ρ
bs=3	RTX 5080	19.70 ± 3.46	0.61	39%
bs=3	Tesla T4	110.04 ± 3.28	0.00	0%
bs=4	RTX 5080	22.79 ± 3.88	0.28	28%
bs=4	Tesla T4	146.61 ± 4.16	0.00	0%

The $5.8\times$ latency ratio demonstrates hardware-dependent boundary location. Even on T4, where configs are far from 20 ms, σ remains 3–4 ms, confirming CPU jitter is intrinsic to the quantization path, not to threshold proximity.

VII. Why Fixed Safety Margins Fail

All margins produce statistically indistinguishable accuracy (overlapping confidence intervals across 10 seeds). Margins penalize stable GPU-resident configs near the threshold while being insufficient for volatile CPU-resident ones ($\sigma = 3.46$ ms

TABLE IV: Fixed Margin Performance (10 seeds, budget=25)

Method	Accuracy	Waste	Configs
Random	0.760 ± 0.009	14.4%	25.0
TPE	0.758 ± 0.010	13.6%	25.0
TBA	0.759 ± 0.010	13.2%	25.0
TBA + 1 ms	0.758 ± 0.010	11.6%	25.0
TBA + 3 ms	0.758 ± 0.010	11.6%	25.0
TBA + 5 ms	0.758 ± 0.010	11.6%	25.0

requires $\delta > 4$ ms). The fundamental problem: the appropriate margin depends on execution device and architecture—information the optimizer lacks.

VIII. Discussion

A. Practical Implications

1. Detect silent device switches. Check `next(model.parameters()).device` after quantization. If it returns `cpu` when GPU was intended, the switch occurred.

2. Repeat borderline CPU evaluations. For CPU-resident configs within ± 5 ms of the threshold, 3–5 evaluations using median p95 provide reliable assessment. GPU-resident configs need only single-shot evaluation.

3. Report execution device alongside latency. Benchmarks should annotate GPU (CUDA events) vs. CPU (wall-clock) timing.

4. Profile on target hardware. The $5.8\times$ shift means feasibility boundaries are hardware-specific.

B. Attempted Optimizer-Level Fix

We investigated Adaptive Confidence-Gated Feasibility (ACGF), which re-evaluates borderline configs with re-evaluations counting against the trial budget. Across 80 runs (8 variants $\times$ 10 seeds, budget=25), ACGF triggered 8 times, changed zero decisions. At practical budgets, borderline configs are too rare for adaptive re-evaluation to be cost-effective. The fix belongs at the evaluation protocol level, not the optimizer level.

C. Limitations

Single primary GPU. Main results are from one RTX 5080 laptop. T4 cross-check validates persistence of CPU jitter.

Windows-specific metrics. `psutil` does not expose dynamic CPU frequency or temperature on Windows. Linux instrumentation may enable finer attribution. The stochasticity may be partially OS-specific.

PyTorch-specific. Other frameworks (TensorRT, ONNX Runtime CUDA EP) may run INT8 on GPU. Our findings apply to PyTorch's `quantize_dynamic` (FBGEMM backend), the most common dynamic quantization path in the ecosystem.

Limited architectures. We study 5 models. Modern transformers and diffusion models may show different jitter profiles.

IX. Related Work

Constrained optimization with noisy constraints. BO under unknown constraints [3] and chance-constrained formulations [4] model constraint uncertainty but assume stationary, uniform noise. Our finding that noise is device- and architecture-dependent adds a structural dimension.

Hyperparameter optimization. Optuna [2] treats constraints as deterministic. TBA [1] handles crashes but treats non-crashed evaluations as deterministic.

PyTorch quantization. Documentation [5] notes CPU-only execution via FBGEMM but does not surface this as a deployment reliability concern. We study the feasibility implications of this device switch.

CPU inference variability. Systems research documents CPU variability from scheduling and power management [9]. We connect this to ML deployment, where CPU jitter ($\sigma \approx 3.5$ ms) is comparable to constraint boundary proximity.

GPU variability. Prior work studies GPU variability in training [8]. We distinguish GPU-side variability (insufficient to flip feasibility) from CPU-side variability (39% flip rates).

ML serving. Clipper [10], InferLine [11], INFaaS [12] address SLAs but not mixed-device execution in configuration search.

X. Conclusion

Deployment search spaces contain a hidden device heterogeneity: dynamic INT8 quantization silently switches inference from GPU to CPU, creating stochastic feasibility boundaries for CPU-resident configurations only. GPU inference is deterministic; CPU inference under dynamic quantization is not. The effect is architecture-dependent (MobileNetV2 volatile, ResNet-18 stable) and hardware-specific (boundaries relocate across GPUs).

Our recommendation: track execution device, repeat borderline CPU measurements 3–5 times, and do not apply uniform safety margins across mixed-device search spaces.

References

[1] C. Lysenstøen, "Feasible-first exploration for constrained ML deployment optimization in crash-prone hierarchical search spaces," 2026. arXiv preprint (under review).

[2] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. KDD*, 2019.

[3] J. R. Gardner, M. J. Kusner, Z. Xu, K. Q. Weinberger, and J. P. Cunningham, "Bayesian optimization with inequality constraints," in *Proc. ICML*, 2014.

[4] A. Ben-Tal, L. El Ghaoui, and A. Nemirovski, *Robust Optimization*. Princeton University Press, 2009.

[5] PyTorch Contributors, "Dynamic quantization," PyTorch Documentation, 2024. https://pytorch.org/docs/stable/quantization.html

[6] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. CVPR*, 2018.

[7] J. Howard, "Imagenette," 2019. https://github.com/fastai/imagenette.

[8] Z. Jia, M. Maggioni, B. Staber, and D. P. Scarpazza, "Dissecting the NVIDIA Volta GPU architecture via microbenchmarking," Technical report, 2018.

[9] T. Li, A. R. Lebeck, and D. J. Sorin, "Quantifying process variation in nanoscale technologies," in *Proc. ISCA*, 2006.

[10] D. Crankshaw *et al.*, "Clipper: A low-latency online prediction serving system," in *Proc. NSDI*, 2017.

[11] D. Crankshaw *et al.*, "InferLine: Latency-aware provisioning and scaling for prediction serving pipelines," in *Proc. SoCC*, 2020.
[12] F. Romero, Q. Li, N. J. Yadwadkar, and C. Kozyrakis, "INFaaS: Automated model-less inference serving," in *Proc. ATC*, 2021.

APPENDIX

Of 25 selected configurations, 5 were excluded due to consistent crashes (4 `torch.compile` failures from Python 3.14 JIT incompatibility, 1 INT8 fatal error). All 20 remaining configurations completed their full schedules.

TABLE V: Configuration Counts and Repeat Breakdown

Category	**Count**
Selected for probe	25
Excluded (crashes)	5
Analyzed (valid)	20
GPU-resident (FP32/FP16)	14
CPU-resident (INT8 dynamic)	6
Repeat breakdown	
Base probe (20 configs $\times$ 50 reps)	1,000
Flipper deep-dive (2 configs $\times$ 200 reps)	400
Cross-model probe (3 configs $\times$ 100 reps)	300
CPU-metrics rerun (2 configs $\times$ 100 reps)	200
Pilot runs (2 configs $\times$ 4 reps)	8
Total measurement trials	**2,150***

*Some configurations appear in multiple experiment phases. Per-config analysis aggregates all available repeats.

All code, raw JSONL data, and analysis scripts: https://github.com/Chrislysen/Constrained-ML-Deployment (branch `boundary-probe`, commit `a68ad42`).

Three commands reproduce all results:

1) `python select_boundary_configs.py` — select configs
2) `python probe_runner.py` — run evaluations
3) `python analyze_boundary.py` — generate figures/stats